WriteWise - AI Writing Coach Browser Extension

Product Requirements Prompt (PRP) v1.0

Context & Business Layer

What We're Building

A Chrome browser extension that provides **real-time, goal-oriented writing suggestions** to help users improve their communication skills across Gmail, Slack, Teams, LinkedIn, and Twitter. The extension processes everything **locally** using lightweight AI models and tracks progress over time.

Core Value Proposition

- **Contextual Writing Improvement**: Suggestions appear as users type, without breaking workflow
- **Goal-Oriented Personalization**: Feedback aligns with user's communication goals (confidence, clarity, warmth, etc.)
- **Privacy-First**: All processing happens locally, no data leaves user's device
- **Progress Tracking**: Measurable improvement over time with analytics

Business Context

- **Target Market**: Individual professionals, academics, and personal brand builders
- **Monetization**: Freemium model (50 suggestions/month free, $9.99/month premium)
- **Competition**: Grammarly, ProWritingAid (we differentiate through goal-based personalization and local processing)
- **Success Metrics**: 70% 7-day retention, 15% freemium conversion within 60 days

---

Technical Architecture Overview

Stack Selection

Frontend/Extension:

- **Chrome Extension Manifest v3**
- **React 18** with TypeScript for popup and dashboard
- **Tailwind CSS** + **shadcn/ui** for consistent UI components
- **Vite** for build tooling and hot reload

Backend/API:

- **FastAPI** (Python 3.11+) for REST API
- **SQLAlchemy 2.0** with **PostgreSQL** for data persistence
- **JWT tokens** for authentication
- **Pydantic v2** for data validation
- **uvicorn** for ASGI server

AI/ML:

- **Transformers.js** for local LLM inference in browser
- **ONNX runtime** for optimized model execution
- **Quantized Llama 2 7B** or similar lightweight model
- **Web Workers** for background processing

Database:

- **PostgreSQL 15+** for user data, goals, and analytics
- **Redis** for session management and rate limiting
- **IndexedDB** (browser) for local extension data

DevOps:

- **Docker** + **Docker Compose** for local development
- **pytest** for Python testing
- **Vitest** for JavaScript/React testing
- **GitHub Actions** for CI/CD

Project Structure

```
writewise/
├── extension/                # Chrome Extension
│   ├── manifest.json         # Extension manifest v3
│   ├── src/
│   │   ├── background/       # Service worker
│   │   │   ├── service-worker.ts
│   │   │   └── llm-worker.ts
│   │   ├── content-scripts/  # Platform integrations
│   │   │   ├── universal-detector.ts
│   │   │   ├── gmail.ts
│   │   │   ├── slack.ts
│   │   │   ├── teams.ts
│   │   │   ├── linkedin.ts
│   │   │   └── twitter.ts
│   │   ├── popup/            # Extension popup
```

```
│   │   ├── Popup.tsx
│   │   └── components/
│   ├── dashboard/          # Full dashboard page
│   │   ├── Dashboard.tsx
│   │   └── components/
│   ├── shared/             # Shared utilities
│   │   ├── storage.ts
│   │   ├── ai-engine.ts
│   │   ├── analytics.ts
│   │   └── api-client.ts
│   └── components/         # Shared UI components
├── public/
│   ├── icons/
│   └── models/             # Local AI models
├── package.json
├── vite.config.ts
└── tailwind.config.js
├── backend/                # FastAPI Backend
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── auth.py     # JWT authentication
│   │   │   │   ├── users.py    # User management
│   │   │   │   ├── goals.py    # Goal management
│   │   │   │   ├── analytics.py # Progress tracking
│   │   │   │   └── suggestions.py # Suggestion history
│   │   │   └── deps.py         # Dependencies
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   └── database.py
│   │   ├── models/             # SQLAlchemy models
│   │   │   ├── user.py
│   │   │   ├── goal.py
│   │   │   ├── suggestion.py
│   │   │   └── analytics.py
│   │   ├── schemas/            # Pydantic schemas
│   │   │   ├── user.py
│   │   │   ├── goal.py
│   │   │   ├── suggestion.py
│   │   │   └── analytics.py
│   │   └── services/           # Business logic
│   │       ├── auth_service.py
│   │       ├── goal_service.py
│   │       └── analytics_service.py
│   ├── tests/
```

```
│   ├── requirements.txt
│   ├── pyproject.toml
│   └── Dockerfile
├── shared/                 # Shared types/constants
│   └── types.ts
├── docs/                   # Documentation
├── docker-compose.yml
├── .env.example
└── README.md
```

Implementation Blueprint

Phase 1: Core Infrastructure (Week 1-2)

Task 1: Set up Chrome Extension Foundation

CREATE `extension/manifest.json`:

```json
```

```
{
  "manifest_version": 3,
  "name": "WriteWise - AI Writing Coach",
  "version": "1.0.0",
  "description": "Goal-oriented writing improvement with privacy-first local AI",
  "permissions": [
    "storage",
    "activeTab",
    "scripting"
  ],
  "host_permissions": [
    "https://mail.google.com/*",
    "https://*.slack.com/*",
    "https://teams.microsoft.com/*",
    "https://www.linkedin.com/*",
    "https://twitter.com/*"
  ],
  "background": {
    "service_worker": "dist/background/service-worker.js"
  },
  "content_scripts": [
    {
      "matches": ["https://mail.google.com/*"],
      "js": ["dist/content-scripts/gmail.js"]
    }
  ],
  "action": {
    "default_popup": "popup.html"
  },
  "web_accessible_resources": [{
    "resources": ["dist/dashboard.html"],
    "matches": ["<all_urls>"]
  }]
}
```

CREATE `extension/src/shared/types.ts`:

```typescript
```

```
export interface WritingGoal {
  id: string
  category: 'professional' | 'personal' | 'academic'
  primary: string
  customizations?: Record<string, any>
}

export interface SuggestionData {
  id: string
  originalText: string
  suggestedText: string
  rationale: string
  platform: string
  goalCategory: string
  confidence: number
}

export interface UserProfile {
  id: string
  name?: string
  email: string
  goals: WritingGoal
  createdAt: Date
}

export interface ProgressMetrics {
  suggestionAcceptanceRate: number
  editFrequency: number
  streakDays: number
  platformBreakdown: Record<string, number>
}
```

Task 2: Backend API Foundation

CREATE `backend/app/main.py`:

```
python
```

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.v1 import auth, users, goals, analytics
from app.core.database import engine
from app.models import user, goal, suggestion, analytics as analytics_model

# Create tables
user.Base.metadata.create_all(bind=engine)
goal.Base.metadata.create_all(bind=engine)

app = FastAPI(title="WriteWise API", version="1.0.0")

# CORS for Chrome Extension
app.add_middleware(
    CORSMiddleware,
    allow_origins=["chrome-extension://*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router, prefix="/api/v1/auth", tags=["auth"])
app.include_router(users.router, prefix="/api/v1/users", tags=["users"])
app.include_router(goals.router, prefix="/api/v1/goals", tags=["goals"])
app.include_router(analytics.router, prefix="/api/v1/analytics", tags=["analytics"])
```

CREATE backend/app/core/security.py:

```python
```

```python
from datetime import datetime, timedelta
from jose import JWTError, jwt
from passlib.context import CryptContext
from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

def verify_token(token: str):
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        return payload.get("sub")
    except JWTError:
        return None
```

Phase 2: AI Engine & Content Detection (Week 2-3)

Task 3: Local AI Processing Engine

CREATE `extension/src/shared/ai-engine.ts`:

```typescript
```

```
import { SuggestionData, WritingGoal } from './types'

class LocalAIEngine {
  private model: any = null
  private isLoaded = false

  async initialize() {
    if (this.isLoaded) return

    // Load quantized model using Transformers.js
    const { pipeline } = await import('@xenova/transformers')
    this.model = await pipeline('text2text-generation', 'Xenova/flan-t5-base')
    this.isLoaded = true
  }

  async analyzeTone(text: string, goal: WritingGoal): Promise<string> {
    await this.initialize()

    const prompt = this.buildPrompt(text, goal)
    const result = await this.model(prompt, { max_length: 100 })
    return result[0].generated_text
  }

  async generateSuggestion(
    text: string,
    goal: WritingGoal,
    platform: string
  ): Promise<SuggestionData | null> {
    const analysis = await this.analyzeTone(text, goal)

    if (!this.needsImprovement(analysis)) {
      return null
    }

    const suggestion = await this.generateImprovement(text, goal, analysis)

    return {
      id: crypto.randomUUID(),
      originalText: text,
      suggestedText: suggestion.text,
      rationale: suggestion.rationale,
      platform,
      goalCategory: goal.category,
      confidence: suggestion.confidence
    }
  }
```

```
  private buildPrompt(text: string, goal: WritingGoal): string {
    const goalPrompts = {
      'confidence': 'Analyze if this text sounds confident and assertive',
      'clarity': 'Analyze if this text is clear and easy to understand',
      'warmth': 'Analyze if this text sounds warm and approachable'
    }

    return `${goalPrompts[goal.primary] || goalPrompts.clarity}: "${text}"`
  }

  private needsImprovement(analysis: string): boolean {
    // Simple heuristic - in production, use more sophisticated analysis
    const improvementKeywords = ['could be', 'might', 'unclear', 'weak']
    return improvementKeywords.some(keyword => analysis.toLowerCase().includes(keyword))
  }

  private async generateImprovement(text: string, goal: WritingGoal, analysis: string) {
    const improvementPrompt = `Improve this text to be more ${goal.primary}: "${text}"`
    const result = await this.model(improvementPrompt, { max_length: 150 })

    return {
      text: result[0].generated_text,
      rationale: `Made more ${goal.primary} based on analysis`,
      confidence: 0.8
    }
  }
}

export const aiEngine = new LocalAIEngine()
```

Task 4: Universal Text Detection

CREATE `extension/src/content-scripts/universal-detector.ts`:

```
typescript
```

```typescript
import { aiEngine } from '../shared/ai-engine'
import { SuggestionUI } from '../components/SuggestionUI'

export class UniversalTextDetector {
  private activeElements = new WeakMap<HTMLElement, SuggestionUI>()
  private debounceTimers = new WeakMap<HTMLElement, NodeJS.Timeout>()

  constructor(private platform: string) {
    this.initializeDetection()
  }

  private initializeDetection() {
    // Detect text inputs, textareas, and contenteditable elements
    const selectors = [
      'input[type="text"]',
      'textarea',
      '[contenteditable="true"]',
      '[role="textbox"]'
    ]

    document.addEventListener('input', this.handleInput.bind(this))
    document.addEventListener('focus', this.handleFocus.bind(this), true)
  }

  private async handleInput(event: Event) {
    const target = event.target as HTMLElement
    if (!this.isWritingElement(target)) return

    // Clear existing timeout
    const existingTimer = this.debounceTimers.get(target)
    if (existingTimer) {
      clearTimeout(existingTimer)
    }

    // Set new timeout for analysis
    const timer = setTimeout(() => {
      this.analyzeText(target)
    }, 2000) // 2 second delay

    this.debounceTimers.set(target, timer)
  }

  private async handleFocus(event: FocusEvent) {
    const target = event.target as HTMLElement
    if (!this.isWritingElement(target)) return
```

```
  // Initialize suggestion UI for this element
  if (!this.activeElements.has(target)) {
    const suggestionUI = new SuggestionUI(target, this.platform)
    this.activeElements.set(target, suggestionUI)
  }
}

private isWritingElement(element: HTMLElement): boolean {
  const tagName = element.tagName.toLowerCase()
  const isInput = tagName === 'input' && element.getAttribute('type') === 'text'
  const isTextarea = tagName === 'textarea'
  const isContentEditable = element.contentEditable === 'true'

  return isInput || isTextarea || isContentEditable
}

private async analyzeText(element: HTMLElement) {
  const text = this.extractText(element)
  if (text.length < 10) return // Skip very short text

  try {
    const userGoals = await this.getUserGoals()
    const suggestion = await aiEngine.generateSuggestion(
      text,
      userGoals,
      this.platform
    )

    if (suggestion) {
      const ui = this.activeElements.get(element)
      ui?.showSuggestion(suggestion)
    }
  } catch (error) {
    console.error('Analysis failed:', error)
  }
}

private extractText(element: HTMLElement): string {
  if (element.tagName.toLowerCase() === 'input' || element.tagName.toLowerCase() === 'textarea') {
    return (element as HTMLInputElement).value
  }
  return element.textContent || ''
}

private async getUserGoals() {
  const { goals } = await chrome.storage.local.get(['goals'])
  return goals || { category: 'professional', primary: 'clarity' }
```

```
  }
}
```

Phase 3: Platform-Specific Integration (Week 3-4)

Task 5: Gmail Integration

CREATE `extension/src/content-scripts/gmail.ts`:

```typescript
```

```
import { UniversalTextDetector } from './universal-detector'

class GmailIntegration extends UniversalTextDetector {
  constructor() {
    super('gmail')
    this.initializeGmailSpecific()
  }

  private initializeGmailSpecific() {
    // Gmail uses dynamic content, so we need to watch for changes
    const observer = new MutationObserver(() => {
      this.attachToComposeBoxes()
    })

    observer.observe(document.body, {
      childList: true,
      subtree: true
    })

    // Initial attachment
    setTimeout(() => this.attachToComposeBoxes(), 1000)
  }

  private attachToComposeBoxes() {
    // Gmail compose box selectors
    const composeSelectors = [
      '[contenteditable="true"][aria-label*="Message"]',
      '[contenteditable="true"][role="textbox"]',
      'div[contenteditable="true"][dir="ltr"]'
    ]

    composeSelectors.forEach(selector => {
      document.querySelectorAll(selector).forEach(element => {
        if (!this.isAlreadyAttached(element as HTMLElement)) {
          this.attachSuggestionUI(element as HTMLElement)
        }
      })
    })
  }

  private isAlreadyAttached(element: HTMLElement): boolean {
    return element.hasAttribute('data-writewise-attached')
  }

  private attachSuggestionUI(element: HTMLElement) {
    element.setAttribute('data-writewise-attached', 'true')
```

```
    // The universal detector will handle the rest
  }
}

// Initialize when DOM is ready
if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', () => new GmailIntegration())
} else {
  new GmailIntegration()
}
```

Task 6: Suggestion UI Component

CREATE `extension/src/components/SuggestionUI.tsx`:

```typescript
```

```tsx
import React, { useState } from 'react'
import { createRoot } from 'react-dom/client'
import { SuggestionData } from '../shared/types'

interface SuggestionPopupProps {
  suggestion: SuggestionData
  onAccept: () => void
  onDismiss: () => void
  onRefine: (tone: string) => void
}

const SuggestionPopup: React.FC<SuggestionPopupProps> = ({
  suggestion,
  onAccept,
  onDismiss,
  onRefine
}) => {
  const [showRefine, setShowRefine] = useState(false)

  return (
    <div className="writewise-popup bg-white border border-gray-200 rounded-lg shadow-lg p-4 max-w-sm">
      <div className="mb-3">
        <h3 className="font-semibold text-gray-800 text-sm">Suggestion</h3>
        <p className="text-xs text-gray-600 mt-1">{suggestion.rationale}</p>
      </div>

      <div className="mb-3">
        <div className="bg-gray-50 rounded p-2 mb-2">
          <p className="text-xs font-medium text-gray-700">Original:</p>
          <p className="text-sm text-gray-800">{suggestion.originalText}</p>
        </div>
        <div className="bg-blue-50 rounded p-2">
          <p className="text-xs font-medium text-blue-700">Suggested:</p>
          <p className="text-sm text-blue-800">{suggestion.suggestedText}</p>
        </div>
      </div>

      <div className="flex gap-2">
        <button
          onClick={onAccept}
          className="flex-1 bg-blue-600 text-white text-xs px-3 py-1.5 rounded hover:bg-blue-700"
        >
          Accept
        </button>
        <button
          onClick={() => setShowRefine(!showRefine)}
```

```
        className="flex-1 bg-gray-100 text-gray-700 text-xs px-3 py-1.5 rounded hover:bg-gray-200"
      >
        Refine
      </button>
      <button
        onClick={onDismiss}
        className="bg-gray-100 text-gray-700 text-xs px-3 py-1.5 rounded hover:bg-gray-200"
      >
        ×
      </button>
    </div>

    {showRefine && (
      <div className="mt-3 pt-3 border-t border-gray-200">
        <p className="text-xs text-gray-600 mb-2">Adjust tone:</p>
        <div className="flex gap-1">
          {['more formal', 'more casual', 'more direct', 'more diplomatic'].map(tone => (
            <button
              key={tone}
              onClick={() => onRefine(tone)}
              className="text-xs bg-gray-100 text-gray-700 px-2 py-1 rounded hover:bg-gray-200"
            >
              {tone}
            </button>
          ))}
        </div>
      </div>
    )}
  </div>
  )
}

export class SuggestionUI {
  private container: HTMLDivElement | null = null
  private root: any = null

  constructor(private targetElement: HTMLElement, private platform: string) {
    this.createContainer()
  }

  private createContainer() {
    this.container = document.createElement('div')
    this.container.className = 'writewise-suggestion-container'
    this.container.style.cssText = `
      position: absolute;
      z-index: 10000;
      display: none;
```

```
    `
    document.body.appendChild(this.container)
    this.root = createRoot(this.container)
  }

  showSuggestion(suggestion: SuggestionData) {
    if (!this.container) return

    const rect = this.targetElement.getBoundingClientRect()
    this.container.style.display = 'block'
    this.container.style.top = `${rect.bottom + window.scrollY + 5}px`
    this.container.style.left = `${rect.left + window.scrollX}px`

    this.root.render(
      <SuggestionPopup
        suggestion={suggestion}
        onAccept={() => this.acceptSuggestion(suggestion)}
        onDismiss={() => this.dismissSuggestion()}
        onRefine={(tone) => this.refineSuggestion(suggestion, tone)}
      />
    )

    // Add highlighting to original text
    this.highlightText()
  }

  private highlightText() {
    this.targetElement.style.border = '2px solid #3b82f6'
    this.targetElement.style.borderRadius = '4px'
  }

  private removeHighlight() {
    this.targetElement.style.border = ''
    this.targetElement.style.borderRadius = ''
  }

  private acceptSuggestion(suggestion: SuggestionData) {
    // Replace text in target element
    if (this.targetElement.tagName.toLowerCase() === 'input' ||
        this.targetElement.tagName.toLowerCase() === 'textarea') {
      (this.targetElement as HTMLInputElement).value = suggestion.suggestedText
    } else {
      this.targetElement.textContent = suggestion.suggestedText
    }

    // Track acceptance
    this.trackSuggestionAction(suggestion, 'accepted')
```

```
    this.hide()
  }

  private dismissSuggestion() {
    // Track dismissal without tracking specific suggestion
    this.hide()
  }

  private async refineSuggestion(suggestion: SuggestionData, tone: string) {
    // Generate refined suggestion
    const refinedSuggestion = await this.generateRefinedSuggestion(suggestion, tone)
    this.showSuggestion(refinedSuggestion)
  }

  private async generateRefinedSuggestion(suggestion: SuggestionData, tone: string): Promise<SuggestionData>
    // This would use the AI engine to refine the suggestion
    return {
      ...suggestion,
      suggestedText: `${suggestion.suggestedText} (${tone})`,
      rationale: `${suggestion.rationale} - adjusted to be ${tone}`
    }
  }

  private async trackSuggestionAction(suggestion: SuggestionData, action: string) {
    // Store in local storage for analytics
    const history = await chrome.storage.local.get(['suggestionHistory']) || { suggestionHistory: [] }
    history.suggestionHistory.push({
      ...suggestion,
      action,
      timestamp: new Date().toISOString()
    })
    await chrome.storage.local.set({ suggestionHistory: history.suggestionHistory })
  }

  private hide() {
    if (this.container) {
      this.container.style.display = 'none'
    }
    this.removeHighlight()
  }
}
```

Phase 4: User Management & Progress Tracking (Week 4-5)

Task 7: User Authentication API

CREATE `backend/app/api/v1/auth.py`:

python

```python
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session
from app.core.database import get_db
from app.core.security import create_access_token, verify_token, pwd_context
from app.models.user import User
from app.schemas.user import UserCreate, UserLogin, Token
from typing import Optional

router = APIRouter()
security = HTTPBearer()

@router.post("/register", response_model=Token)
async def register(user_data: UserCreate, db: Session = Depends(get_db)):
    # Check if user exists
    existing_user = db.query(User).filter(User.email == user_data.email).first()
    if existing_user:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="User already exists"
        )

    # Create new user
    hashed_password = pwd_context.hash(user_data.password)
    user = User(
        email=user_data.email,
        name=user_data.name,
        hashed_password=hashed_password
    )
    db.add(user)
    db.commit()
    db.refresh(user)

    # Generate token
    access_token = create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}

@router.post("/login", response_model=Token)
async def login(user_data: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == user_data.email).first()
    if not user or not pwd_context.verify(user_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials"
        )
```

```
    access_token = create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: Session = Depends(get_db)
) -> User:
    token = credentials.credentials
    email = verify_token(token)
    if not email:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token"
        )

    user = db.query(User).filter(User.email == email).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User not found"
        )

    return user
```

Task 8: Progress Analytics Service

CREATE `backend/app/services/analytics_service.py`:

```
python
```

```python
from sqlalchemy.orm import Session
from sqlalchemy import func, and_
from app.models.suggestion import Suggestion
from app.models.user import User
from datetime import datetime, timedelta
from typing import Dict, List
import statistics

class AnalyticsService:
    def __init__(self, db: Session):
        self.db = db

    def get_user_progress(self, user_id: str, days: int = 30) -> Dict:
        end_date = datetime.utcnow()
        start_date = end_date - timedelta(days=days)

        suggestions = self.db.query(Suggestion).filter(
            and_(
                Suggestion.user_id == user_id,
                Suggestion.created_at >= start_date
```